Cairo and Starknet

A learning group for ZK and SNARK application development

Daniel Szego
In: https://www.linkedin.com/in/daniel-szego/

Respectful collaboration starts here

www.lfdecentralizedtrust.org/code-of-conduct

Agenda

- *Logistics*
- *Zero knowledge proofs - summary*
- *SNARK vs STARK*
- *High level flow of zero knowledge programming*
- *Cairo*
- *Starknet*
- *Demo*
- *Challenge*
- *Links, Resources, Literature*
- *Q&A*

Logistics

Every month or two months

Similar structure, similar logistics + ***guest lectures***

Theory:

- Look tables,
- Zk variations: STARKs, FRI
- recursive SNARKs

Programming / platforms:

- ZoKrates, Leo
- Zinc / ZKSync

Applications:

- ZK bridges
- ZK Virtual Machines
- ZK Identity / verifiable credential - presentations
- Voting

Discord channel:

https://discord.com/channels/905194001349627914/1329201532628898036

Meetup.com:

https://www.meetup.com/lfdt-hungary/events/305634614/

Repo with all the contents:

https://github.com/LF-Decentralized-Trust-labs/zk-learning-group

Zero knowledge proofs - summary

Verifiable computation with cryptographic proof

Scarce resource:
- limited verification time: no re-execution
- storage limits: smaller proof
- limited information

Computation: arithmetic circuit : *C(x, w) -> F*
- *x* public input
- *w* private input, witness
- high level computation
- arithmetic circuit
- polynomials

Prover algorithm: *P (pp, x, w) -> proof*

Verifier algorithm: *V (vp, x, proof) -> accept / reject*

Interactive / non-interactive proofs

SNARK vs STARK

SNARK

- **Succinct:**

  Small in size and fast to verify:

  constant - sublinear: few hundred bytes

  Proof generation might be resource intensive

- **Non-interactive:**

  No interaction between the prover and the verifier, the prover publishes the proof and the verifier either accepts or rejects

- **Argument of Knowledge:**

  The prover knows a fact to “prove”

No quantum resistance: elliptic curve cryptography, bilinear pairing - Schor algorithm as quantum attack

STARK

- **Scalable:**

  Generating proof is fast (quasilinear, (O(nlog {2}n)))

  Verification is polylogarithmic

  log2 - few hundred kbytes

  Different computational model

- **Transparent:**

  No trusted setup

- **Argument of knowledge:**

  The prover knows a fact to “prove”

Quantum resistance: hash functions - Groover algorithms as quantum attack

High level flow of zero knowledge programming

Computation:

- “High” level programs
- Input-output, program instructions

Arithmetization:

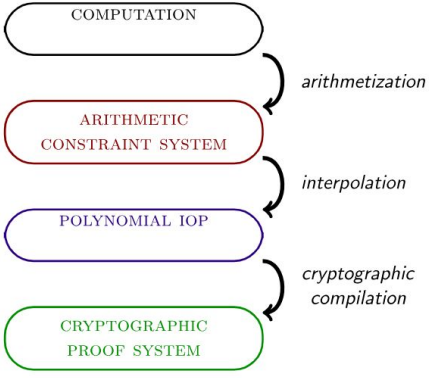
- Arithmetic constraint system
- Equations on finite fields
- Constraint system solution: valid computation
- *AIR (Algebraic Intermediate Representation)*

Polynoms

- Polynomial IOP (interactive oracle proof)

Proof system:

- Polynomial commitments



Cairo

Rust based high level programming language

Designed to implement provable programs: STARK proof

Mostly for written for using in Starknet, separate usage is possible

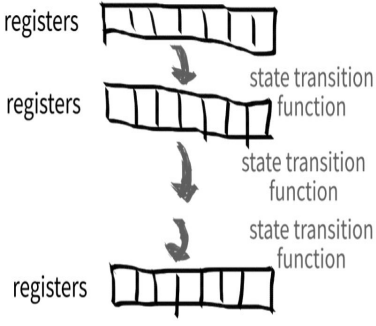
Cairo = CPU **AIR** (Algebraic Intermediate Representation)

Sierra (Safe Intermediate Representation)

CASM: Cairo Assembly

CairoVM: Cairo Virtual Machine: specific virtual machine optimized for running provable software with STARK proofs.

- Turing complete
- Stark optimized



https://www.cairo-lang.org/

Cairo

Rust based language: valuables, mutability, ownership, borrowing

Rust style syntax, functions, expressions

Data types:
- simple: field, int, string, boolean
- complex: tuples, arrays, structures, dictionaries, enums

Control flow: if / else / loop / while

Generics, closures, functional language features

Advanced features:
- hashes
- macros / procedural macros for starknet programs
- smart pointers
- custom data structures

https://www.starknet.io/cairo-book/ch01-00-getting-started.html

Starknet

Ethereum L2 scaling network

Zero knowledge rollup system

General purpose turing complete smart contracts implemented in Cairo

Account abstraction:
- Advanced wallet features
- Social login
- Auto-fee
- Biometric security

Native token: **STRK**
https://coinmarketcap.com/currencies/starknet-token/

https://www.starknet.io/

Starknet

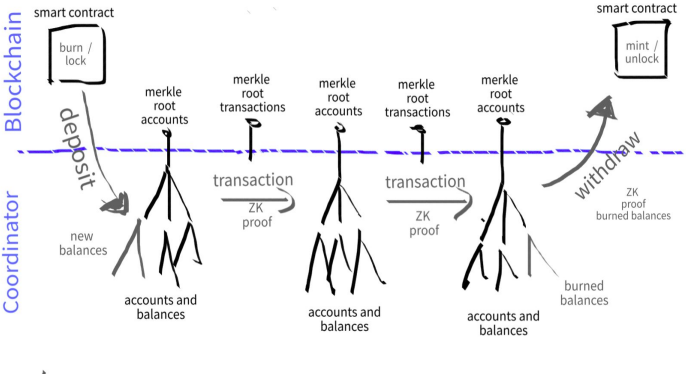
Transactions: Cairo smart contract code executed off-chain by CairoVM

Off-chain state: world-state stored by the the sequencer with on-chain integrity

Sequencers: getting transactions, collecting and generating blocks (set of transactions)

Provers: creating STARK proof on blocks

Ethereum verification: verifies proofs on the Ethereum network



Demo

Cairo playground

https://www.cairo-lang.org/cairovm/

Challenge

Build a simple (9,9) sudoku puzzle with Cairo

Links, Resources, Literature

The Cairo Book
https://www.starknet.io/cairo-book/#the-cairo-book

Cairo – a Turing-complete STARK-friendly CPU architecture
https://eprint.iacr.org/2021/1063.pdf

Cairo programming language
https://www.cairo-lang.org/

Starknet tutorials
https://www.starknet.io/tutorials/

Cairo playground
https://www.cairo-lang.org/cairovm/

Happy Hunting for the SNARK :)

Q & A

Daniel Szego

In: https://www.linkedin.com/in/daniel-szego/